

Day 39: Sort Product Prices

Day 39 — Sort Product Prices

1. Day Number

Day 39

2. Topic Name

List Sorting — `sort()` Method

Today you will learn how to arrange items inside a Python list in **sorted order**.

For product prices, we usually want:

Low price → High price

3. Connection

You already have product prices stored in a list.

Today you will learn how to **sort those prices from low to high**.

Example:

Before sorting:
[450, 120, 300, 99]

After sorting:
[99, 120, 300, 450]

This is useful in applications such as:

- shopping websites
 - product catalogs
 - price comparison systems
 - e-commerce search results
-

4. Important Topics

Today focus on three ideas:

- **List of numbers**
- `sort()`
- **Sorted order**

Example list:

prices = [500, 200, 350, 100]

Here each item represents a product price.

5. Foundational Notes

What is sorting?

Sorting means arranging values in a particular order.

For numbers, two common orders are:

```
Ascending:  
100, 200, 300, 400  
  
Descending:  
400, 300, 200, 100
```

Today's problem uses **ascending order**:

```
low → high
```

Python sort() method

Python lists provide the `sort()` method.

Basic syntax:

```
list_name.sort()
```

For example:

```
numbers = [8, 2, 5, 1]  
  
numbers.sort()
```

After sorting, `numbers` becomes:

```
[1, 2, 5, 8]
```

sort() changes the original list

This is an important point.

Suppose:

```
numbers = [30, 10, 20]
```

After:

```
numbers.sort()
```

the original list itself changes:

```
Before:  
[30, 10, 20]
```

```
After:  
[10, 20, 30]
```

You do not need to create another list for today's problem.

sort() sorts numbers from low to high by default

You can simply use:

```
prices.sort()
```

Python automatically arranges the numbers in ascending order.

Conceptually:

```
[900, 200, 500, 100]  
  
    sort()  
  
[100, 200, 500, 900]
```

Be careful with this

sort() modifies the list and does **not** return the sorted list.

So this idea is incorrect:

```
result = prices.sort()
```

because result will not contain your sorted prices.

For today's exercise, think:

```
Create prices  
    ↓  
Sort prices  
    ↓  
Print prices
```

6. Easy Example

Here is a simple example unrelated to the final problem:

```
scores = [70, 40, 90, 60]  
  
scores.sort()  
  
print(scores)
```

Output:

```
[40, 60, 70, 90]
```

Notice:

```
70  
40  
90  
60
```

became:

```
40  
60  
70  
90
```

The smallest value comes first.

7. Problem Statement

Create a list containing several **product prices**.

Then:

1. Sort the prices from **low to high**
2. Print the sorted list

Example idea:

```
Original prices:  
  
[450, 100, 800, 250]  
  
Sort them  
  
Result:  
  
[100, 250, 450, 800]
```

Use Python's **sort()** method.

8. Concepts Used

You will use:

List

Stores multiple prices.

Concept:

```
prices = [...]
```

Numbers

Each price can be an integer or floating-point number.

For example:

```
100
250
499
```

sort()

Changes the order of the values inside the list.

Concept:

```
prices.sort()
```

Function

To follow a functional approach, your sorting logic can be placed inside a function.

Conceptually:

```
function
    receive prices
    sort prices
    return or display result
```

print()

Used to display the sorted prices.

9. Thought Process

Before writing Python, think through the problem.

Step 1 — What data do I have?

You have several product prices.

Example:

```
450
120
700
250
```

Multiple related values suggest using a:

```
list
```

Step 2 — What do I need to do?

You need to arrange the prices:

```
smallest → largest
```

Step 3 — Which Python feature can do this?

A list provides:

```
sort()
```

Step 4 — What happens after sorting?

The original list becomes sorted.

Example:

Before

```
[450, 120, 700, 250]
```

↓ sort

After

```
[120, 250, 450, 700]
```

Step 5 — What should I display?

Print the sorted price list.

Overall thinking:

```
Product prices
↓
Store in list
↓
Pass list to function
↓
Sort using sort()
↓
Print sorted prices
```

10. Pseudocode

```
START
```

```
CREATE a function to sort product prices
```

```
    SORT the price list using sort()
```

```
    RETURN the sorted price list

CREATE a list of product prices

CALL the sorting function

PRINT the sorted prices

END
```

Notice that pseudocode describes the logic without giving you the complete Python answer.

11. Suggested Solving Approach — Functional Approach

Use a small function responsible for sorting the prices.

Think of the structure like this:

```
Main program
  |
  | product prices
  v
+-----+
| sort_prices() |
|               |
| sort the list |
| return the result |
+-----+
  |
  v
Sorted prices
  |
  v
Print
```

Your function should have one simple responsibility:

| **Sort the list of product prices.**

This keeps the program easier to understand.

Conceptually:

```
Input
[400, 100, 300]

    ↓

sort_prices(...)

    ↓
```

Output
[100, 300, 400]

12. Easy Edge Cases

Edge Case 1 — Empty List

Example:

[]

Sorting an empty list is fine.

The result remains:

[]

There is nothing to rearrange.

Edge Case 2 — Already Sorted List

Example:

[100, 200, 300, 400]

After sorting:

[100, 200, 300, 400]

Nothing changes because the list is already in the correct order.

Edge Case 3 — Duplicate Prices

Example:

[300, 100, 300, 200]

After sorting:

[100, 200, 300, 300]

Duplicate values are **not removed**.

Sorting only changes the order.

This distinction is important:

Sorting ≠ Removing duplicates

13. Expected Input

For today's problem, you can create the list directly rather than asking the user for input.

For example, imagine the program starts with:

```
Product prices:
```

```
450  
120  
700  
250  
120
```

Represented conceptually as:

```
[450, 120, 700, 250, 120]
```

No keyboard input is required unless you want extra practice.

14. Expected Output

For the example above:

```
Sorted product prices:
```

```
[120, 120, 250, 450, 700]
```

Another example:

```
Prices:
```

```
[999, 250, 500, 100]
```

Expected sorted result:

```
[100, 250, 500, 999]
```

15. Hint Only

Think about these three pieces:

1. Create a list containing prices.
2. Inside your function, use:
`list_name.sort()`
3. Print the list after the function has sorted it.

Remember:

```
sort()
```

changes the **existing list**.

Your final program flow should look roughly like:

```
prices
  ↓
sorting function
  ↓
sort()
  ↓
sorted prices
  ↓
print
```

Your task: try implementing `sort_prices()` yourself without looking for a complete solution.